Machine Learning Project 2026/2027 Part II (Classification) - Prediction of Harmful Algal Bloom (HAB) Events and Pseudo-nitzschia Concentration

Alexandre Bernardino, Margarida Silveira, Catarina Barata, Ayman Bnoussaad

November 1, 2026

1 Foreword

The Machine Learning project is composed of one regression and two classification problems. This document presents the two classification problems.

At the end of each classification task, the group must deliver a code package in a specific format described in Section 6.

The code to be delivered will be evaluated automatically. To allow automatic processing, the code must comply with some requirements on the allowed libraries and corresponding versions, as described in Section 10.

Some functions and objects of Python libraries will not be permitted because they hide important machine learning aspects that students must be exposed to. The set of non permitted functions and objects is described in Section 7.

Students must also comply with fairness rules on group work and use of AI. Templates are provided about these aspects and must be filled and included in the deliverables, as described in Section 9.

Failure to comply with any of the above rules will invalidate the deliverable. To help students verify the rules, validation code is provided to check the code compliance to the rules.

Finally, code will be checked for plagiarism. Detected cases will be penalized according to university rules.

2 Introduction

Coastal monitoring activities leverage satellite data to track the presence of harmful algae and the concentration of undesirable bacteria, such as the diatom Pseudo-nitzschia. The goal of these activities is not only to assess the quality of the water, but also to provide alerts that may result in fishing bans for certain areas.

Satellite data, which come in the form of hyperspectral data, are acquired at specific time intervals (e.g, daily). The different hyperspectral bands can be converted into measurements, conveying information about different biological processes that occur at the surface of the sea, like the chlorophyll-a concentration that was explored in the previous regression task. Together with other meteorological measurements, such as the upwelling index or the sea temperature, these data can be used to predict the occurrence of undesirable events.

In this second part of the project, you will explore how the various measurements can be used to develop two predictors:

1. A model that classifies the occurrence of a Harmful Algal Bloom (HAB) event given a set of meteorological and satellite derived metrics; and
2. A model that discretizes the concentration of the Pseudo-nitzschia (PN) bacteria into a set of possible classes.

The **first task is a binary classification problem**, while the **second will be a multiclass one**. In both settings, you will need to cope with different real-world challenges present in classification datasets, namely: i) class imbalance; ii) missing features; iii) variables with different ranges; and iv) irrelevant and/or highly correlated features. Additionally, you will need to compare different classification algorithms, optimize them, and select the most suitable one for each task.

3 Task 1 - Prediction of HAB Occurrence

The first task of the project consists of predicting the daily occurrence of an HAB event (0 - no occurrence, 1 - occurrence) given a set of spatial, meteorological, and hyperspectral-derived observations. The provided training data comprises the features presented in Table 1, collected across non-uniform time intervals from 2013 to 2020. Moreover, these samples have also been acquired from multiple spatial locations in the North of Portugal.

Table 1: Variables in the Dataset for Task 1 - Prediction of HAB Occurrence

Column	**Description**	**Type / Units/Range**	% **Missing**
Date	Sampling time	datetime	0%
Day / Month / Year	Calendar components of Date	int	0%
Season	Spring/Summer/Fall/Winter	category	0%
DOY	$\sin(2\pi \times \text{DayOfYear}/365)$	float [-1,1]	0%
spatial_cluster	Spatial groups	int {0,1,2,3,4,5}	0%
SST	Sea Surface Temperature	float, °C	0%
UI	Upwelling Index	float	0%
CHL	Chlorophyll-a concentration	float, mg/m^3	46.00%
CHL_uncertainty	Uncertainty estimate for CHL	float	46.00%
HAB	TARGET	int {0,1}	0%

The data is available in the file `classification_task1_train.xlsx`. Each row corresponds to a day and each column to one data type. Similarly to the regression task, you will need to use the `Pandas` library to read the `.xlsx` into a `DataFrame` and process it. Some of the columns contain redundant information (*e.g*, "Date and "Day", "Month", "Year"), but were included to facilitate the navigation. "CHL" and "CHL_uncertainty" were derived from the hyperspectral data and are missing for 46% of the days. The "spatial_cluster" is a derived metric, obtained by applying a clustering method to the original spatial coordinates of the sampled points and taking into consideration their proximity to river streams. Figure 1 exemplifies the cluster locations. Finally, HAB corresponds to the target variable (output).

3.1 Task Goal and Challenges

You must optimize and compare at least two classification algorithms in order to predict the occurrence of a HAB event given the measurements reported in Table 1 for a specific day. While developing your approach, consider the characteristics of the training dataset and how to handle them. Some examples are: i) understanding whether your classes are balanced; and ii) how you can handle the missing observations for CHL and CHL_uncertainty. We also encourage the students to start by computing additional statistics of the dataset, in order to identify further challenges.

The teaching team will evaluate the developed method using only complete daily samples from the years 2021 to 2023. This means that while the training data contains missing observations for CHL and CHL_uncertainty, all samples of the test set are complete.

3.2 Assessment

The primary evaluation metric for the selected classification algorithm will be the **Area Under the Curve** (AUC), computed from the values of sensitivity/recall and 1-specificity across different decision

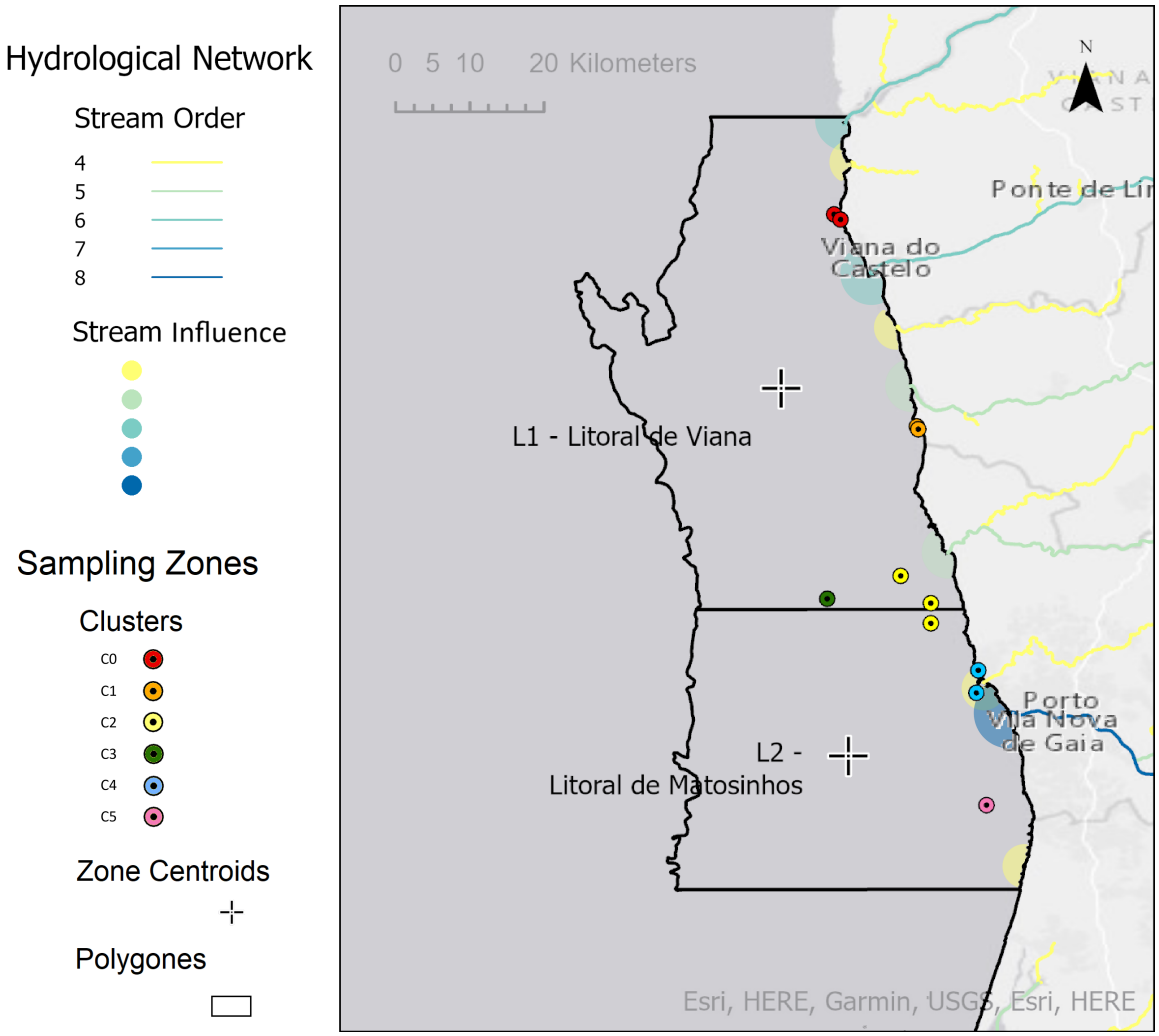


Figure 1: Spatial Clusters - Located along the Portuguese North Coast.

thresholds. Nevertheless, the students are encouraged to explore additional metrics, namely those directly derived from the confusion matrix (*e.g.* recall, precision, f1-score).

The format of the test file used by the teaching team to validate your model will be similar to the one you are provided for training and selecting the best approach. However, the column that matches the TARGET ('HAB') will be missing. Thus, the code you include in the `my_model.py` must comprise a function to parse the `.xlsx` file and create the data matrix X. **The output of your `predict` function should be the predicted HAB event class for each of the available days (0 - no occurrence or 1 - occurrence).**

4 Task 2 - Prediction of Pseudo-nitzschia Concentration

In this second task you will develop a classification algorithm to predict the concentration class (PN) of the Pseudo-nitzschia bacteria for a certain day. Concentrations take continuous values, however they have been discretized them into 4 bins, according to the following international rules:

- Negligible: PN = 0
- Low: 0 < PN < 1,000
- Medium: 1,000 ≤ PN < 10,000
- Bloom: PN ≥ 10,000

Thus, the goal of this task will be to develop a multiclass method that predicts one of the four classes. To achieve this goal, the variables listed in Table 2 will be made available.

Table 2: Variables in the Dataset for Task 2 - PN Concentration

Column	**Description**	**Type / Units or Range**
Date	Sampling time	datetime
Day / Month / Year	Calendar components of Date	int
Season	Spring/Summer/Fall/Winter	category
DOY	$\sin(2\pi \times \text{DayOfYear}/365)$	float [-1,1]
spatial_cluster	Spatial groups	category {0,1,2,3,4,5}
SST	Sea Surface Temperature	float, °C
UI	Upwelling Index	float
CHL	Chlorophyll-a concentration	float, mg/m^3
CHL_uncertainty	Uncertainty estimate for CHL	float
DIATO	Diatom biomass fraction	float
DINO	Dinoflagellate biomass fraction	float
HAPTO	Haptophyte biomass fraction	float
GREEN	Green algae biomass fraction	float
PROKAR	Prokaryote biomass fraction	float
PROCHLO	Prochlorococcus biomass fraction	float
MICRO / NANO / PICO	Phytoplankton size-class fractions (micro/nano/picoplankton)	float
*_uncertainty (for DIATO...PICO)	Uncertainty estimates paired with each fraction-group column	float
PN	TARGET	int $\{0, 1, 2, 3\}$

Besides the daily measurements (identified as $*$_lag0), the dataset also contains some of the readings for the prior 45 days ($*$_lag#), offering longitudinal information about the evolution of the different quantities. This information was collected over several days at non-uniform temporal intervals and multiple spatial locations between 2013 and 2020. The file provided to the students is titled `classification_task2_train.xlsx`.

4.1 Task Goal and Challenges

You must optimize and compare at least two classification algorithms in order to predict the concentration of PN given the measurements listed in Table 2 and their lags, for a specific day. As in the case of Task 1, you must consider the challenging characteristics of the training dataset and how to handle them.

The teaching team will evaluate the developed method using samples from the years 2021 to 2023. In this case, the test set will also exhibit missing data. Therefore, the implemented best_model must account for this.

4.2 Assessment

The primary evaluation metric for the selected classification algorithm will be the **Balanced Accuracy** (BAcc), computed macro-averaging the recall values of the different classes. Nevertheless, the students are encouraged to explore additional metrics.

The format of the test file used by the teaching team to validate your model will be similar to the one you are provided for training and selecting the best approach. However, the column that matches the TARGET ('PN') will be missing. Thus, the code you include in the `my_model.py` must comprise a function to parse the `.xlsx` file and create the data matrix X. **The output of your `predict` function must be a class label $\in$ {0,1,2,3} for each of the days.**

5 Provided Files

- `ML_LAB_PART_2.pdf`: this handout;
- `classification_task1_train.xlsx` and `.csv`: released data for classification task 1;
- `classification_task2_train.xlsx` and `.csv`: released data for classification task 2;
- `validate_submission_2.py` to validate your submission prior to uploading it on Fénix.

6 Submission format

Submit one ZIP file through the Fenix project interface. The system identifies your group from the submission folder name. Do not place the required files inside an extra top-level folder. The ZIP root must contain:

```
best_model/
    my_model.py
    model.pkl
    utils.py
training_code/
    train.py
    <other files used in training>
declarations/
    group_declaration_of_honor.pdf
    student_1_ai_declaration.pdf
    student_2_ai_declaration.pdf
```

The required `best_model/` directory contains the selected fitted model and all code executed during prediction. The file `my_model.py` defines the grading interface, `model.pkl` stores the fitted predictive model together with plain numerical parameters, and `utils.py` contains any auxiliary prediction functions imported by `my_model.py`. All three filenames are required. Additional runtime files are not allowed outside this directory.

The required `training_code/` directory must contain a main script named `train.py`. It must reproduce the complete training workflow from the released raw data: reading and sorting the data, constructing lagged variables, defining the chronological validation split, calculating every normalization or transformation parameter, selecting and fitting the final model, and saving `best_model/model.pkl`.

Other source modules, configuration files, and a short README may be included in this directory. A notebook may be supplied as supplementary material, but it does not replace `train.py`.

The `declarations/` directory must contain the three PDFs shown above. `group_declaration_of_honor.pdf` must use the group template supplied by the teaching team for the previous assignment (Part1). Each student must complete one individual AI-use declaration, stored as `student_1_ai_declaration.pdf` or `student_2_ai_declaration.pdf`. The order of the students must be the same as in the group declaration. The submission simulation verifies that all three files exist and are valid PDFs.

Do not submit datasets, virtual environments, absolute paths, cached files, or downloaded model binaries. Keep prediction-time files under `best_model/` and training-only files under `training_code/`.

File `best_model/my_model.py` must define exactly the following public entry points:

```
def load_model(model_path):
    """Return␣the␣fitted␣predictor␣and␣plain␣numerical␣state."""

def predict(model,day_and_lag_features_df):
    """Return␣a␣finite␣NumPy␣array␣with␣shape␣(1,)."""
```

The automated assessment tool loads the model once. For each test date it calls `predict` with:

- the loaded model
- `day_and_lag_features_df`: A Pandas DataFrame with the prediction dates in 2021 to 2023 together with all the features made available for the training data.

Your `predict` function must explicitly create its data matrix X, apply the stored numerical normalization parameters, apply any output transformation, preserve input DataFrames, and return the expected output for each of the classification tasks. It must not retrain, access the network, read undeclared files, use future rows, or print large outputs.

7 Explicit Code Rules

All preprocessing, feature construction, target transformation, and output post-processing must be written explicitly in `train.py`, `my_model.py`, or your own functions in submitted source files. You may use ordinary NumPy and Pandas operations, for example, to calculate a mean and standard deviation, standardize a matrix, create lagged columns, or other explicit and legible operations. You may also use a library implementation of the **classification algorithms**, such neural networks, SVM, decision trees, etc.

Do not use a pre-defined library object or convenience function that hides a sequence of preprocessing or post-processing operations. Classes and functions from `sklearn.preprocessing` are permitted, provided that any fitted preprocessing is learned only from the appropriate training data and the same fitted transformation is applied at prediction time. The submission must not use `Pipeline`, `make_pipeline`, `ColumnTransformer`, imputers (e.g, `impute` from scikit), feature selectors, feature-extraction or decomposition objects, `TransformedTargetRegressor`, or calibration objects. These restrictions apply during both training and prediction.

The file `model.pkl` may contain the fitted predictive estimator, fitted `sklearn.preprocessing` objects used to transform input features, and plain state such as dictionaries, lists, strings, numbers, and NumPy arrays. A fitted scaler may store its learned means and scales; other required parameters may be saved as plain values. Fit preprocessing objects only on the appropriate training data, and apply those same fitted objects to the corresponding features, in the same column order, during prediction. The file must not contain a `Pipeline`, `ColumnTransformer`, `TransformedTargetRegressor`, or another object prohibited by the submission rules. The construction of the data matrix X and output post-processing must remain explicit in the submitted source code. The supplied submission simulation and professor-side grader inspect both the Python source and the loaded artifact for these forbidden constructs. This automated check covers ordinary uses; the teaching team may also inspect the source when necessary.

Before zipping, run:

```
python validate_submissions_2.py SUBMISSION_PATH classification_task#_train.xlsx
```

A successful submission simulation checks the fixed `best_model/`, `training_code/`, and `declarations/` structure; the required files; forbidden transformation imports and serialized objects; importability of `my_model.py` and its sibling `utils.py`; callable signatures; model loading; and prediction shape. It does not run the training workflow or reveal the hidden test score.

8 Reproducibility checklist

Before submitting, confirm that:

- the ZIP has no extra directory level;
- `best_model/` contains `my_model.py`, `model.pkl`, and `utils.py`;
- `training_code/` contains the complete Python training and preprocessing source;
- `training_code/train.py` is the entry point for reproducing the fitted model;
- `declarations/group_declaration_of_honor.pdf` follows the supplied group template;
- `declarations/` contains `student_1_ai_declaration.pdf` and `student_2_ai_declaration.pdf`;
- all required file and directory names match exactly, including case;
- `predict` works after starting Python from a different directory;
- all preprocessing parameters were learned from training data and saved;
- every transformation is implemented explicitly in submitted source code;
- `model.pkl` contains no pipeline, scaler, transformer, or post-processing object;
- date sorting is explicit;
- predictions have shape `(1,)`, must be a value in {0,1} for task 1 and {0,1,2,3} for task 2;
- the model does not depend on notebook state, environment variables, or local absolute paths;
- a clean environment can install the listed dependencies; and
- the supplied submission simulation reports `PASS` for the final submission folder.

9 Academic integrity and permitted tools

Follow the course policy on collaboration and external tools. You may use the standard numerical and machine-learning packages available in the declared environment, subject to the submission rules. Library predictive estimators and `sklearn.preprocessing` tools for input features are permitted. Fitted preprocessing must be learned from training data only and applied consistently during prediction. Library pipelines, imputers, feature selectors, target-transformation wrappers, and output post-processing wrappers remain prohibited. Lag construction and any output post-processing must be implemented explicitly in the submitted source code. Your team is responsible for understanding every submitted preprocessing and modelling step. The teaching team may inspect the source code and reproduce the submitted model from `training_code/`.

The `declaration_templates/` directory contains two editable Word templates. Each group must complete and sign one copy of `group_declaration_of_honor_template.docx`. Each student must separately complete and sign one copy of `individual_ai_use_declaration_template.docx`, including the AI tools used and the purpose and extent of their use. If no AI tool was used, state this explicitly in the individual declaration rather than leaving the corresponding field blank.

Export the completed declarations to PDF and include them in the submitted `declarations/` directory using the exact filenames `group_declaration_of_honor.pdf`, `student_1_ai_declaration.pdf`, and `student_2_ai_declaration.pdf`. The student order must match the order in the group declaration. Do not submit blank templates or editable Word files in place of the required signed PDFs.

10 Recommended environment and dependencies

To ensure compatibility with the provided code students must use Python 3.9 through 3.13; Python 3.11 is recommended. The project includes a `requirements.txt` file containing the supported dependencies:

```
numpy>=1.24,<3
pandas>=2.0,<3
scipy>=1.10,<2
openpyxl>=3.1,<4
scikit-learn==1.6.1
```

It is advised to use virtual environment with the recommended dependencies:

```
python -m venv .venv
```

On Linux or macOS, activate it with:

```
source .venv/bin/activate
```

On Windows, activate it with:

```
.venv\Scripts\activate
```

The required packages can be installed with:

```
python -m pip install --upgrade pip
python -m pip install --only-binary=:all: -r requirements.txt
```

Students should test their submissions in a clean environment containing only the dependencies listed above. Code that depends on an unlisted package may fail in the automated assessment tools.